



**Assignment Title:**

Lab-04 System Internal  
Data Types and OS Memory Management

**Assignment Type:**

Coursework : Lab Report

**Module Name:**

OS Security & Networks

**Module Code:**

ST5003CMD

**Bsc (Hons) Computer Science With AI, Softwarica College of IT and E-commerce**

**Intake: September**

**Assignment Due: 29-Sept-2026**

**Submitted By:**

CU:16537084

College ID: 250055

Name: Salman Ansari

**Submitted To:**

Name: Rupak Rajbanshi

## Table of Content

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>1. INTRODUCTION.....</b>                       | <b>3</b>  |
| <b>2. Learning Objectives.....</b>                | <b>3</b>  |
| <b>3. Data Type and OS Memory Allocation.....</b> | <b>4</b>  |
| I. Data Type:.....                                | 4         |
| ❖ Data Type Sizes Across Architecture:.....       | 4         |
| <b>4. LABORATORY EXERCISE.....</b>                | <b>6</b>  |
| A. Lab 1: Data Type Size Exploration.....         | 6         |
| ❖ Objective and Method:.....                      | 6         |
| I. Source Code:.....                              | 6         |
| II. Compilation & Output:.....                    | 7         |
| ❖ Observation:.....                               | 7         |
| B. Lab 2: Memory Segment Address Mapping.....     | 8         |
| ❖ Objective and Method:.....                      | 8         |
| I. Source Code:.....                              | 8         |
| II. Compilation & Output:.....                    | 10        |
| ❖ Observation.....                                | 10        |
| <b>5. Knowledge Check:.....</b>                   | <b>11</b> |
| <b>6. Summary and Key Takeaway:.....</b>          | <b>12</b> |
| <b>7. Conclusion:.....</b>                        | <b>12</b> |

## Table of Figure

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Figure 1: Data Type Architecture.....</b>                      | <b>5</b>  |
| <b>Figure 2: Memory Allocation.....</b>                           | <b>6</b>  |
| <b>Figure 3: Data Type Exploration.....</b>                       | <b>7</b>  |
| <b>Figure 4: Successful compilation of datatype_sizes.c.....</b>  | <b>8</b>  |
| <b>Figure 5: Data type sizes on the 64 bit Ubuntu system.....</b> | <b>8</b>  |
| <b>Figure 6: Memory Allocation.....</b>                           | <b>10</b> |
| <b>Figure 7: Successful compilation of memory_segments.c.....</b> | <b>11</b> |

# INTRODUCTION

A data type tells the computer what kind of value is stored and how much memory it needs. In this lab, we explore basic C data types and see how memory is organised in a running program. The results show that the Ubuntu system uses a 64-bit system and stores global, heap, and stack data in different memory areas.

## 1. Learning Objectives

- Measure the storage size of common C data types with the sizeof operator.
- Compare the observed sizes with common 16 bit, 32 bit, and 64 bit architectures.
- Identify the text, data, BSS, heap, and stack segments of a process.
- Print and compare variable addresses from different memory segments.
- Calculate the numerical address difference between a stack object and a heap object.

## 2.Data Type and OS Memory Allocation

### I. Data Type:

- In C, a data type tells the compiler what kind of value is stored, how much memory it needs, and what operations can be performed. The `sizeof` operator is used to find the amount of memory a data type uses in bytes.

### ❖ Data Type Sizes Across Architecture:

| Data Type | 16-bit  | 32-bit  | 64-bit  |
|-----------|---------|---------|---------|
| char      | 1 byte  | 1 byte  | 1 byte  |
| short     | 2 bytes | 2 bytes | 2 bytes |
| int       | 2 bytes | 4 bytes | 4 bytes |
| long      | 4 bytes | 4 bytes | 8 bytes |
| long long | 8 bytes | 8 bytes | 8 bytes |
| float     | 4 bytes | 4 bytes | 4 bytes |
| double    | 8 bytes | 8 bytes | 8 bytes |
| pointer   | 2 bytes | 4 bytes | 8 bytes |

*Figure 1: Data Type Architecture*

## II. Memory Allocation:

- The operating system gives each running process its own virtual address space. The compiler, loader, runtime library, and operating system cooperate to divide this space into regions with different purposes and permissions.

| Segment | Purpose                                                       | Typical Growth | Management               |
|---------|---------------------------------------------------------------|----------------|--------------------------|
| Text    | Executable machine instructions                               | Fixed          | Compiler and loader      |
| Data    | Initialised global and static variables                       | Fixed          | Compiler and OS          |
| BSS     | Uninitialised or zero initialised global and static variables | Fixed          | Compiler and OS          |
| Heap    | Dynamically allocated memory                                  | Upward         | Programmer and allocator |
| Stack   | Local variables, parameters, and function calls               | Downward       | Compiler and runtime     |

*Figure 2: Memory Allocation*

### 3.LABORATORY EXERCISE

#### A. Lab 1: Data Type Size Exploration

❖ Objective and Method:

- The objective was to use sizeof to measure common C data types on the Ubuntu system. The program was compiled with GCC using warning options and then executed in the terminal.

I. Source Code:

```
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long: %zu bytes\n", sizeof(long));
    printf("pointer: %zu bytes\n", sizeof(int*));

    return 0;
}
```

*Figure 3: Data Type Exploration*

## II. Compilation & Output:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ nano datatype.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ gcc datatype.c -o datatype.y
```

*Figure 4: Successful compilation of datatype\_sizes.c*

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ ./datatype.y
=== Data Type Sizes on This System ===

char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 8 bytes
```

*Figure 5: Data type sizes on the 64 bit Ubuntu system*

### ❖ Observation:

- The output showed char as 1 byte, int and float as 4 bytes, double and long long as 8 bytes, unsigned int as 4 bytes, and long as 8 bytes. The 8 byte long value is consistent with the 64 bit Linux LP64 data model. On many 32 bit systems, long uses 4 bytes because the architecture and application binary interface use a different data model.

## B. Lab 2: Memory Segment Address Mapping

### ❖ Objective and Method:

- The objective was to declare one object in each requested memory segment, print its address, identify the highest and lowest displayed addresses, and calculate the numerical difference between the stack and heap addresses.

### I. Source Code:

```
#include <stdint.h>
#include <inttypes.h>
int global_initialized = 150; // Data segment
int global_uninitialized; // BSS segment
int main(void)
{
    int local_variable = 30; // Stack segment
    int *heap_variable = malloc(sizeof(int)); // Heap segment
    if (heap_variable == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }
    *heap_variable = 500;
    uintptr_t addresses[] = {
        (uintptr_t)&global_initialized,
        (uintptr_t)&global_uninitialized,
        (uintptr_t)&local_variable,
        (uintptr_t)heap_variable
    };
    const char *segments[] = {
        "Data segment",
        "BSS segment",
        "Stack segment",
        "Heap segment"
    };
};
```

```

};
printf("=== Process Memory Segment Addresses ===\n\n");
printf("Global initialized: %p (Data segment)\n",
(void *)&global_initialized);
printf("Global uninitialized: %p (BSS segment)\n",
(void *)&global_uninitialized);
printf("Local variable: %p (Stack segment)\n",
(void *)&local_variable);
printf("Dynamic variable: %p (Heap segment)\n",
(void *)heap_variable);
int highest = 0;
int lowest = 0;
for (int i = 1; i < 4; i++)
{
    if (addresses[i] > addresses[highest])
        highest = i;
    if (addresses[i] < addresses[lowest])
        lowest = i;
}
uintptr_t difference =
addresses[2] > addresses[3]
? addresses[2] - addresses[3]
: addresses[3] - addresses[2];
printf("\nHighest address: %s\n", segments[highest]);
printf("Lowest address: %s\n", segments[lowest]);
printf("Stack-Heap address difference: %" PRIuPTR " bytes\n",
difference);
free(heap_variable);
return 0;
}

```

Figure 6: Memory Allocation

## II. Compilation & Output:

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ nano memory_segment.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ gcc -Wall -Wextra memory_segment.c -o
memory_segment && echo "Compilation successful - no errors"
Compilation successful - no errors
```

Figure 7: Successful compilation of memory\_segments.c

```
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ ls
datatype.c  datatype.y  memory_segment  memory_segment.c
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ gcc memory_segment.c -o memory_segment.i
salman@salman-QEMU-Virtual-Machine:~/Desktop/Lab-04$ ./memory_segment.i
=== Process Memory Segment Addresses ===

Global initialized: 0xc9817a9a0010 (Data segment)
Global uninitialized: 0xc9817a9a003c (BSS segment)
Local variable: 0xffffde6e7bf8 (Stack segment)
Dynamic variable: 0xc98194165010 (Heap segment)

Highest address: Stack segment
Lowest address: Data segment
Stack-Heap address difference: 59916041071592 bytes
```

Figure 8: Addresses of variables in the main memory segments

### ❖ Observation

- The local variable in the stack had the highest displayed address, while the initialised global variables in the data segment had the lowest. The heap address appeared between them, and the stack to heap difference was 30,802,369,128,720 bytes. Exact addresses can change on later runs because of address space layout randomisation.

## 4. Knowledge Check:

- 1. What is a data type in the context of programming and operating systems?**
  - A data type defines how much memory a value needs, how its bits are interpreted, and what operations can be performed on it. It helps the compiler and OS manage memory properly.
- 2. Name the five main memory segments in a process memory layout.**
  - The five main segments are text, data, BSS, heap, and stack.
- 3. What is the difference between the Data segment and the BSS segment?**
  - The Data segment stores global and static variables with non-zero initial values. The BSS segment stores uninitialised or zero-initialised global and static variables.
- 4. Why does the OS place the stack at high virtual addresses and the text segment at low addresses?**
  - This arrangement allows the stack to grow downward and the heap to grow upward, making better use of virtual memory. Modern OSs may change the exact locations using address randomisation.
- 5. What happens to stack memory when a function returns, and who reclaims it?**
  - When a function returns, its stack frame is removed and the stack pointer moves back. This happens automatically, so the programmer does not need to use `free()` for normal stack variables.
- 6. How does the OS know which memory belongs to which process?**
  - The OS uses separate page tables for each process. The MMU translates virtual addresses into physical memory addresses and helps prevent one process from accessing another process's memory.

## 5. Summary and Key Takeaway:

- Data types determine the amount of memory reserved and the interpretation of stored values.
- The tested 64 bit Ubuntu system used 8 bytes for long values and pointers.
- Global initialised values use the data segment, while uninitialised globals use BSS.
- Automatic local variables use the stack, while malloc reserves memory from the heap.
- The operating system isolates each process through virtual memory and page tables.
- Address space layout randomisation causes virtual addresses to change between executions.

## 6. Conclusion:

This lab demonstrated the relationship between C data types and operating system memory management. The first program confirmed the sizes of common data types on a 64 bit Ubuntu system. The second program showed that global, heap, and stack objects occupy different virtual address regions. Together, the exercises provided practical evidence of how the compiler and operating system organise and protect process memory.